

PROJECT 5

ACTIVATION FUNCTIONS USING PYTHON

NAME:ATHUL K KOSHY

REG NO:2411021240048

❖ Introduction

Activation functions are mathematical functions used in artificial neural networks to transform the input of a neuron into an output. They introduce non-linearity, which helps neural networks learn complex patterns. In this project, three commonly used activation functions—Sigmoid, ReLU, and Tanh—are implemented using Python.

❖ Objectives

- To implement the Sigmoid activation function using Python.
- To implement the ReLU activation function using Python.
- To implement the Tanh activation function using Python.
- To generate outputs for different input values.
- To visualize and compare the behavior of the three activation functions.

❖ Tools and Libraries Used

- Python
- Google Colab
- NumPy – used for numerical calculations.
- Matplotlib – used for plotting the activation functions.

❖ Activation Functions

▪ Sigmoid

- The Sigmoid function converts an input into a value between 0 and 1. It has a smooth S-shaped curve.

$$\text{➤ } \text{Sigmoid}(x) = 1 / (1 + e^{-x})$$

▪ ReLU

- ReLU (Rectified Linear Unit) returns 0 for negative input values and returns the input itself for positive values.

$$\text{➤ } \text{ReLU}(x) = \max(0, x)$$

▪ Tanh

- The Tanh function produces outputs between -1 and +1. It is also an S-shaped function and is centered around zero.

$$\text{➤ } \text{Tanh}(x) = \tanh(x)$$

❖ Python Implementation

The following screenshot shows the Python implementation of Sigmoid, ReLU, and Tanh, along with the selected input values and their generated outputs.

```
import numpy as np
import matplotlib.pyplot as plt

# Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# ReLU Activation Function
def relu(x):
    return np.maximum(0, x)

# Tanh Activation Function
def tanh(x):
    return np.tanh(x)

# Different input values
x = np.array([-5, -3, -1, 0, 1, 3, 5])

print("Input values:", x)
print("Sigmoid:", sigmoid(x))
print("ReLU:", relu(x))
print("Tanh:", tanh(x))

Input values: [-5 -3 -1  0  1  3  5]
Sigmoid: [0.00669285 0.04742587 0.26894142 0.5          0.73105858 0.95257413
 0.99330715]
ReLU: [0 0 0 0 1 3 5]
Tanh: [-0.9999092  -0.99505475 -0.76159416  0.          0.76159416  0.99505475
 0.9999092 ]
```

❖ Generated Outputs

The input values used in the program were -5, -3, -1, 0, 1, 3, and 5. The program calculated the corresponding outputs for all three activation functions.

Input	Sigmoid	ReLU	Tanh
-5	0.0067	0	-0.9999
-3	0.0474	0	-0.9951
-1	0.2689	0	-0.7616
0	0.5000	0	0.0000
1	0.7311	1	0.7616
3	0.9526	3	0.9951
5	0.9933	5	0.9999

The output values show that Sigmoid remains between 0 and 1, ReLU changes negative values to zero, and Tanh remains between -1 and +1.

❖ Visualization

A range of input values from -10 to 10 was generated using NumPy. Matplotlib was then used to plot the three activation functions on the same graph.

```

# Generate values for the graph
x = np.linspace(-10, 10, 400)

# Calculate activation function outputs
y_sigmoid = sigmoid(x)
y_relu = relu(x)
y_tanh = tanh(x)

# Create the graphs
plt.figure(figsize=(10, 6))

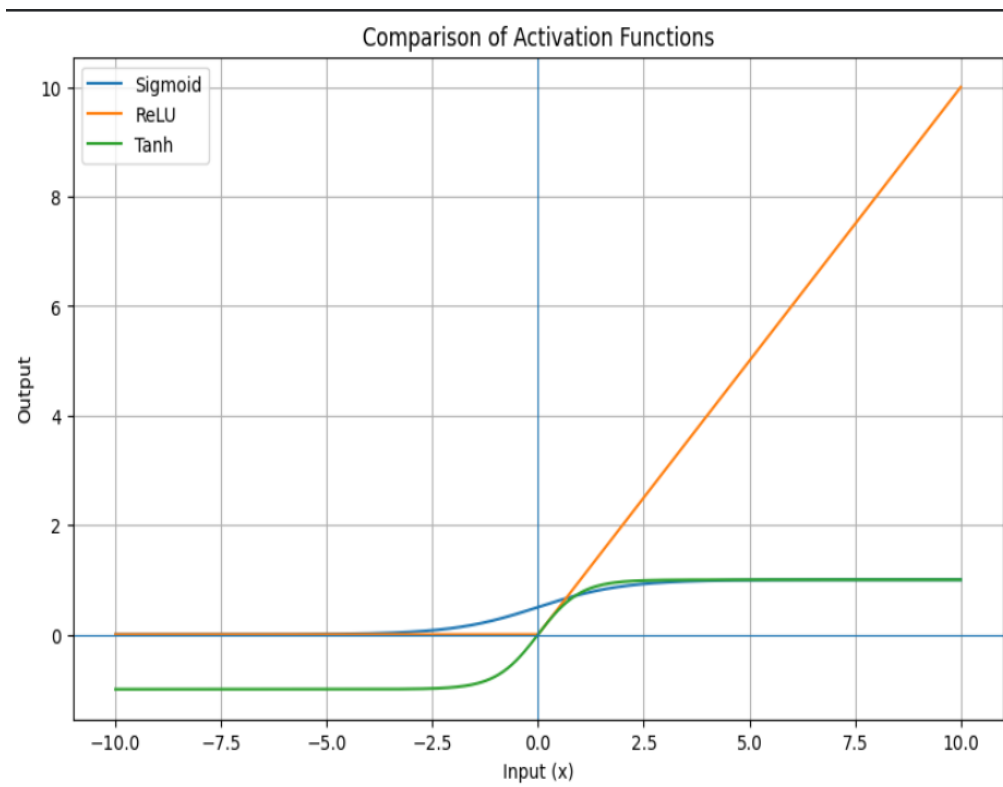
plt.plot(x, y_sigmoid, label="Sigmoid")
plt.plot(x, y_relu, label="ReLU")
plt.plot(x, y_tanh, label="Tanh")

plt.title("Comparison of Activation Functions")
plt.xlabel("Input (x)")
plt.ylabel("Output")
plt.axhline(0, linewidth=0.8)
plt.axvline(0, linewidth=0.8)
plt.legend()
plt.grid(True)

plt.show()

```

Python code used to generate the comparison graph



Comparison of Sigmoid, ReLU, and Tanh

❖ Comparison of Activation Functions

Function	Output Range	Main Behavior	Key Feature
Sigmoid	0 to 1	Smoothly converts inputs	Useful when output is required between 0 and 1
ReLU	0 to ∞	Negative values become 0	Simple and computationally efficient
Tanh	-1 to 1	Produces positive and negative outputs	Centered around zero

❖ Conclusion

In this project, the Sigmoid, ReLU, and Tanh activation functions were successfully implemented using Python. Different input values were tested and the resulting outputs were observed. The functions were also visualized using Matplotlib. The comparison shows that each activation function has a different output range and behavior. This project provides a basic understanding of how activation functions transform inputs in neural networks.